DAA Lab - Experiment 2

Implement divide and conquer based Merge Sort and Quick Sort and compare their performance

Dr. Ayush Kumar Agrawal
Assistant Professor, School of Computer Science
UPES, Dehradun

Objective

- Implement Merge Sort and Quick Sort using divide and conquer.
- Compare their performance for the same input dataset.
- Understand best, average, and worst case complexities.

Theory

- Both Merge Sort and Quick Sort use the **Divide and Conquer** paradigm.
- **Merge Sort:** Divide array into halves, sort recursively, merge results.
- **Quick Sort:** Partition array around a pivot, sort left and right partitions recursively.
- Merge Sort: $O(n \log n)$ in all cases.
- Quick Sort: $O(n \log n)$ on average, $O(n^2)$ in worst case (bad pivot).

Algorithm: Merge Sort

MergeSort(A, low, high)

1. If low ¡ high:
   - mid = (low + high)/2
   - MergeSort(A, low, mid)
   - MergeSort(A, mid+1, high)
   - Merge(A, low, mid, high)

Algorithm: Quick Sort

QuickSort(A, low, high)

1. If low ¡ high:
   - pivot = Partition(A, low, high)
   - QuickSort(A, low, pivot-1)
   - QuickSort(A, pivot+1, high)

C Code - Merge Sort (ONLY FOR REFERENCE)

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void merge(int A[], int low, int mid, int high) {
    int n1 = mid - low + 1;
    int n2 = high - mid;
    int L[n1], R[n2];
    for (int i = 0; i < n1; i++) L[i] = A[low+i];
    for (int j = 0; j < n2; j++) R[j] = A[mid+1+j];
    int i=0, j=0, k=low;
    while (i<n1 && j<n2)
        A[k++] = (L[i] <= R[j]) ? L[i++] : R[j++];
    while (i<n1) A[k++] = L[i++];
    while (j<n2) A[k++] = R[j++];
}
void mergeSort(int A[], int low, int high) {
    if (low < high) {
        int mid = (low+high)/2;
        mergeSort(A, low, mid);
        mergeSort(A, mid+1, high);
        merge(A, low, mid, high);
    }
}
```

C Code - Quick Sort (ONLY FOR REFERENCE)

```
int partition(int A[], int low, int high) {
    int pivot = A[high];
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (A[j] <= pivot) {
            i++;
            int temp = A[i]; A[i] = A[j]; A[j] = temp;
        }
    }
    int temp = A[i+1]; A[i+1] = A[high]; A[high] = temp;
    return (i+1);
}

void quickSort(int A[], int low, int high) {
    if (low < high) {
        int p = partition(A, low, high);
        quickSort(A, low, p-1);
        quickSort(A, p+1, high);
    }
}
```

Main Function (ONLY FOR REFERENCE)

```
int main() {
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    int A[n], B[n];
    printf("Enter %d elements:\n", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &A[i]);
        B[i] = A[i];
    }
    clock_t start, end;
    start = clock();
    mergeSort(A, 0, n-1);
    end = clock();
    double tMerge = ((double)(end-start))/CLOCKS_PER_SEC;
    start = clock();
    quickSort(B, 0, n-1);
    end = clock();
    double tQuick = ((double)(end-start))/CLOCKS_PER_SEC;
    printf("Merge Sort Time: %.6f ms\n", tMerge*1000);
    printf("Quick Sort Time: %.6f ms\n", tQuick*1000);
    return 0;
}
```

Sample Input/Output

Input:

n = 5

38 27 43 3 9

Output:

Merge Sort Time: 0.002 ms

Quick Sort Time: 0.001 ms

Complexity Analysis

- Merge Sort:
  - Best, Avg, Worst: $O(n \log n)$
  - Space: $O(n)$
- Quick Sort:
  - Best, Avg: $O(n \log n)$
  - Worst: $O(n^2)$ (bad pivot)
  - Space: $O(\log n)$ (stack calls)

Observation Table

Input Size	Merge Sort (ms)	Quick Sort (ms)
100	0.10	0.08
1000	0.90	0.75
5000	5.60	4.80

Comparison

- Merge Sort is stable and guaranteed $O(n \log n)$.
- Quick Sort is faster in practice but can degrade to $O(n^2)$.
- Merge Sort needs extra space, Quick Sort is in-place.

Viva Questions

1. Which algorithm is stable, Merge Sort or Quick Sort?
2. Why is Quick Sort usually faster than Merge Sort?
3. How can the worst case of Quick Sort be avoided?
4. Which algorithm is preferred for linked lists?

Conclusion

- Both Merge Sort and Quick Sort follow divide and conquer.
- Merge Sort is predictable and stable, Quick Sort is faster on average.
- Choice depends on dataset and memory constraints.

References

- Cormen et al., *Introduction to Algorithms*, MIT Press.
- Horowitz & Sahni, *Fundamentals of Computer Algorithms*.